WEEK 5

Spark in Depth

Shuffles, Joins & Performance Engineering

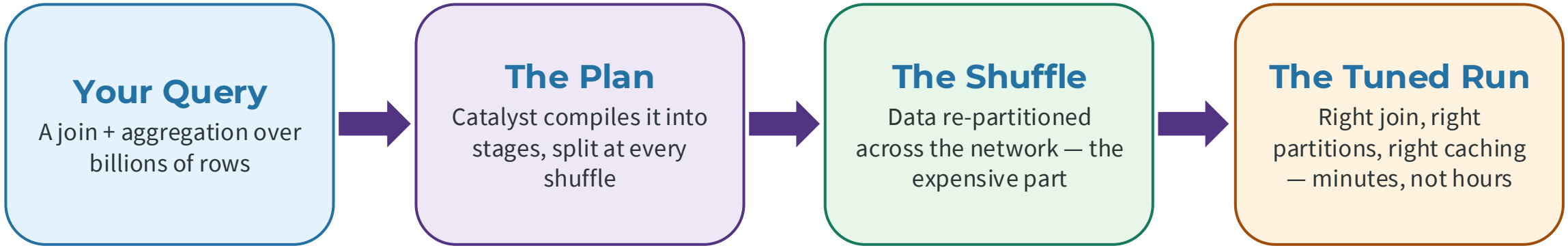
Shuffles
Wide vs. narrow — what really moves across the cluster

Joins
Broadcast, sort-merge & shuffle-hash

Performance
Plans, partitions, caching & skew

CDLE • MEIC/MEIM • ISEL

From a Slow Job to a Fast One



This week is about seeing inside the engine

Same result, 10–100× less time — once you can read what Spark is actually doing and act on the evidence.

NEW TERMS

`Join` combine rows of two tables that share a key value

`Aggregation` many rows → one per group: count, sum, avg…

`Billion` 10^9, short scale (US) · PT: mil milhões, not bilião

Contents

Everything for week 5 — how Spark really executes your job, and how to make it fast with evidence

01 Inside the Engine

The 40-minute join — a scenario
Narrow vs. wide dependencies
Anatomy of a shuffle
Jobs, stages & tasks

02 Plans & Joins

Reading plans: groupBy vs. join
Join strategies: broadcast, sort-merge, shuffle-hash
Choosing — and forcing — the right join
Joins without an equi-key: bucketing, Cartesian

03 Performance Tuning

The Spark UI: where the evidence lives
Partition tuning & Adaptive Query Execution
Caching wisely · data skew & salting
Reading list

04 AI & the Lab

The AI assistant as performance consultant
Lab 5 — the performance clinic
Tooling, FAQs & takeaways

Scenario — The 40-Minute Join

A 2TB × 2MB join that should take a few minutes takes 40 (minutes)

The Job

"Enrich every click with its product category, then save the result."

- Clicks: `≈2 TB / 4 × 10⁹ rows` · ≈500 Bytes per row
- Products: `2 MB / 12,000 rows` · ≈170 Bytes per row
- `clicks.join(products, "product_id")` then `write.parquet(…)`
- **Cluster:** 25 executors (machines) × 8 cores, 56 GB each, 10 Gbit/s network: 200 task slots, ≈4 GB working memory per task; Spark 3.x, AQE off
- On the cluster it runs for **40 minutes** — every task slow, none failing

The Evidence

The Spark UI tells a different story from the code:

- One stage shuffles **1.9 TB** across the network (productID may be null, LZ4, …)
- All 200 join tasks are **equally slow** (~16 min each) and spill ≈1 TB to disk
- **Both tables were shuffled** because products is read via JDBC: Spark had no size, assumed it was huge and chose sort-merge over broadcast
- The cluster is healthy. **The plan is pathological.**

The lesson

A 2 MB table should never force 2 TB across the network. By the end of today you will spot this in the plan — before you press run.

NEW TERMS

`clicks` the click log: one row per click, 2 TB
`products` the catalogue: one row per product, 2 MB
`AQE` Adaptive Query Execution: re-plans at runtime
`Spill` data a task writes to local disk when its memory fills
`LZ4` fast compression Spark applies to shuffle and spill files
`JDBC` reading a table from a database

Narrow vs Wide Dependencies

Two kinds of transformations — one is nearly free, the other pays the network

Narrow — stays local

```
# keep rows
clicks.filter(col("country") == "PT")
# keep columns
clicks.select("user_id", "ts")
# add a column
clicks.withColumn("day", to_date("ts"))
# no network — pipelined
```

Wide — shuffle required

```
# count per key
clicks.groupBy("product_id").count()
# match two tables
clicks.join(products, "product_id")
# sort everything
clicks.orderBy("ts")
# keys must be co-located
```

How the plan shows it

```
clicks.groupBy("product_id") \
      .count().explain()
== Physical Plan - full plan → slide 11
*HashAggregate
+- Exchange
   hashpartitioning(product_id)
# Exchange = shuffle
```

✓ Pipelined in one stage

✗ Cuts the job into stages

✓ Count the Exchanges

*Narrow transformations chain freely inside a stage; every **wide** one inserts an **Exchange**. Count the Exchanges in a plan and you know exactly how many shuffles you will pay for. The plan above: 1 Exchange → 2 stages.*
Slide 4's job: 2 Exchanges (one per join input, AQE off) → 3 stages: scan clicks, scan products, then join + write.

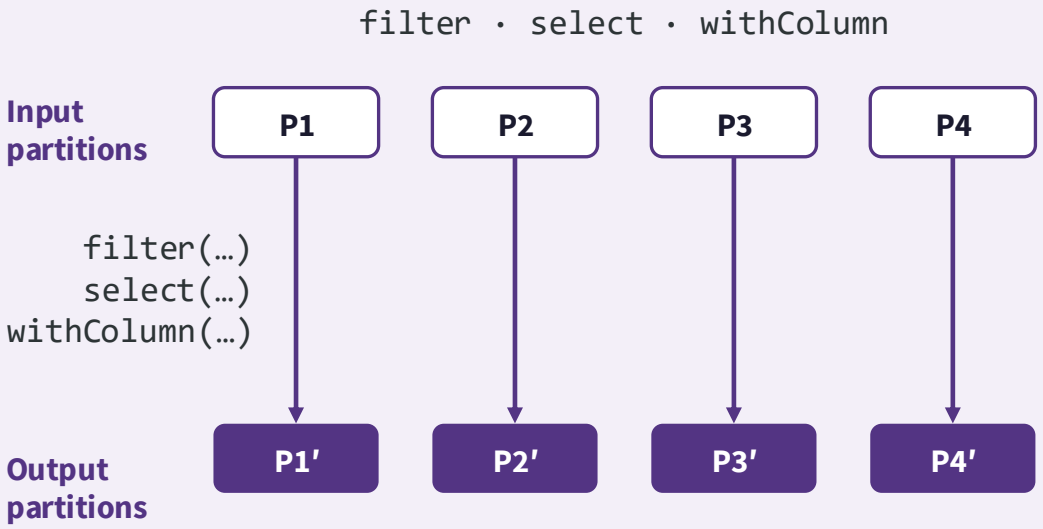
NEW TERMS

- **`withColumn`** add or replace one column
- **`groupBy · orderBy`** group rows by key · sort all rows
- **`col("x")`** refers to a column by name
- **`hashpartitioning(k)`** row → partition hash(k) mod n
- **`Co-located`** on the same executor, so no data moves
- **`to_date`** turns a timestamp into a date

Narrow vs Wide, Drawn

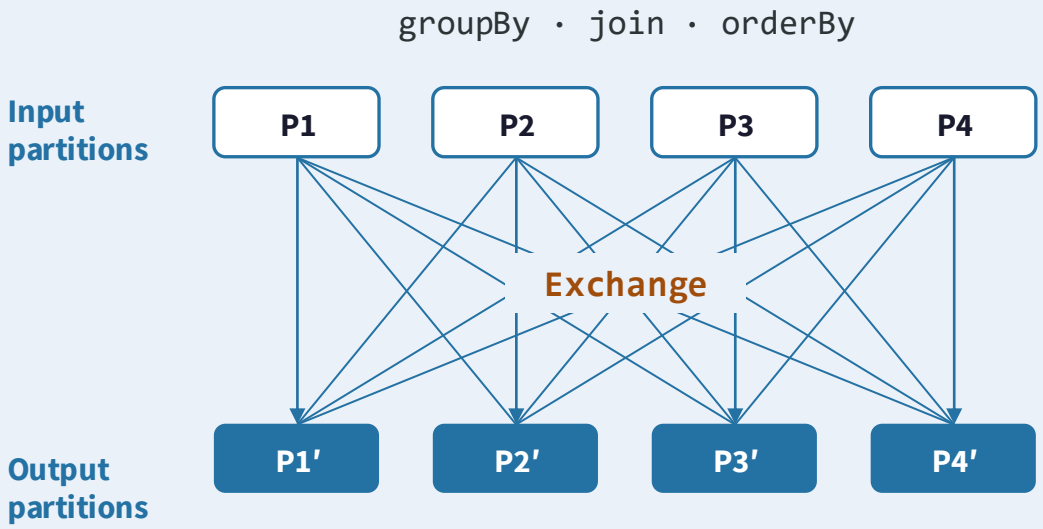
Each box is a partition on one executor; each arrow is a data dependency from input to output

Narrow — 1 → 1, stays on the executor



One task per arrow, no data between executors. P1′ stays in memory; local disk only at an Exchange, storage only at a write.

Wide — many → many, crosses the network



Every output needs rows from every input. Spark inserts an Exchange: a shuffle, and a stage cut.

Rule of thumb: *if the result for one key can need rows from every partition, the dependency is wide and you will pay a shuffle.*

Anatomy of a Shuffle

What actually happens between two stages — map side, the wire, reduce side

1

Map side — write

Each upstream task hash-partitions its output by key and writes sorted shuffle blocks to local disk.

2

The stage barrier

The stage ends here. No downstream task may start until every map task has finished — stragglers stall everyone.

3

Fetch over the wire

Every reduce task pulls its slice from every map task — an all-to-all exchange of M × R block fetches.

4

Reduce side — merge

Fetched blocks are merged and aggregated, spilling to disk when memory runs short; the next stage begins.

Why shuffles dominate cost

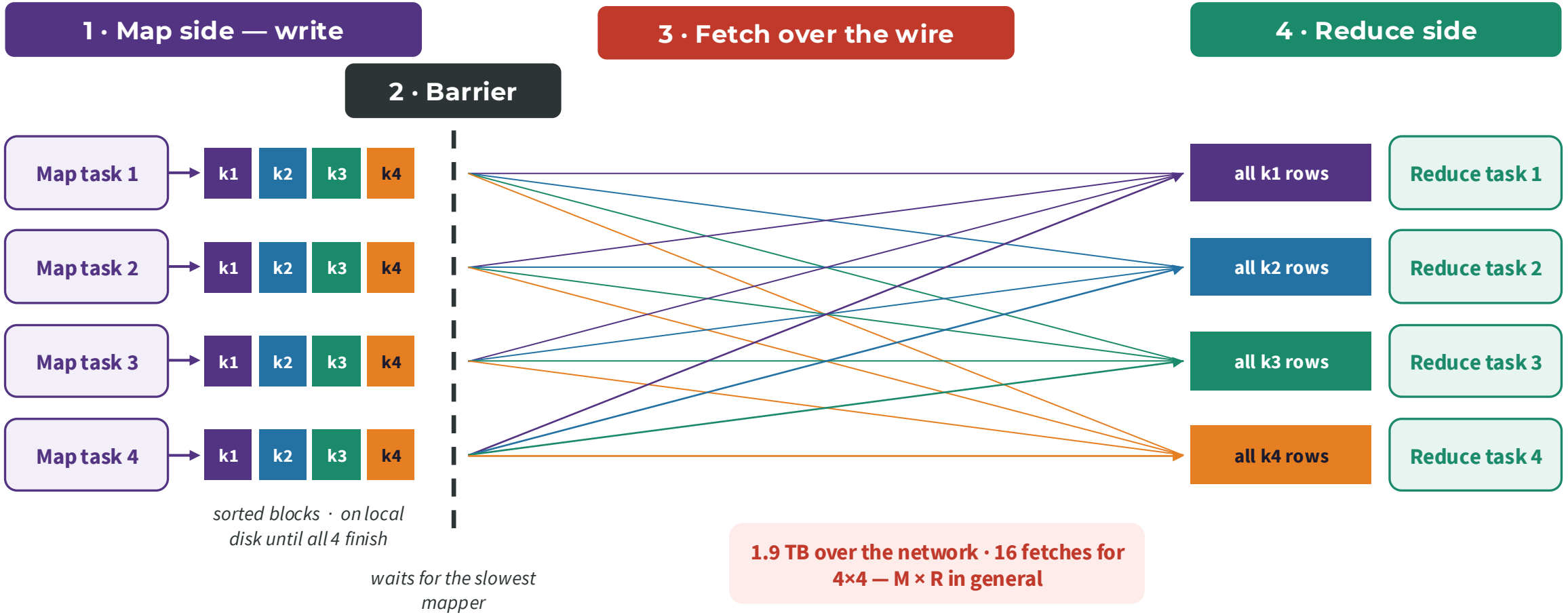
- One operation, four cost centres: serialisation, disk I/O, network transfer and memory pressure.
- You cannot avoid every shuffle — the craft is shuffling **less data, fewer times**.

NEW TERMS

`Upstream` the earlier stage; downstream = the later
`Shuffle block` one map task's output for one reducer
`Straggler` a task far slower than its peers
`Barrier` the point where a stage waits for all tasks
`Spill` write data to disk when memory runs short
`Serialisation` turn objects into bytes to store or send

A Shuffle, Drawn

Stage 3 of the baseline join: mappers write, the barrier waits, reducers fetch from every mapper



Cost lives in three places: *disk writes on the map side, the wait at the barrier, and the M × R fetch — which is why the fix is to move less data, not to add executors.*

Jobs, Stages & Tasks

1 — Job
one per action — e.g. show(), count(), write.parquet(); contains one or more stages

2 — Stage
cut at every shuffle boundary; contains one or more tasks

3 — Task
one per partition, through one stage, run on one executor; the smallest unit

One query, decomposed by Spark

```
# clicks.join(products, "product_id")
#       .write.parquet("enriched_clicks")
Job 1  triggered by the action .write
├─ Stage 1  scan clicks     16,000 tasks
        → Exchange · 1.9 TB
├─ Stage 2  scan products   1 task
        → Exchange · 2 MB
└─ Stage 3  join + write    200 tasks
     ↑ reads both, waits for every task of 1 & 2
# two Exchanges → three stages
```

Why the hierarchy matters

- ✓ An action triggers a job — until then Spark is lazy
- ✓ Stage boundaries are shuffles — fewer shuffles, fewer stages
- ✓ Tasks are the unit of parallelism — and of skew
- ✓ One slow task stalls its entire stage
- ✓ The Spark UI is organised exactly this way

Read a job the way Spark runs it — stages between shuffles, tasks within stages.
That map is your debugging compass for the rest of the semester.

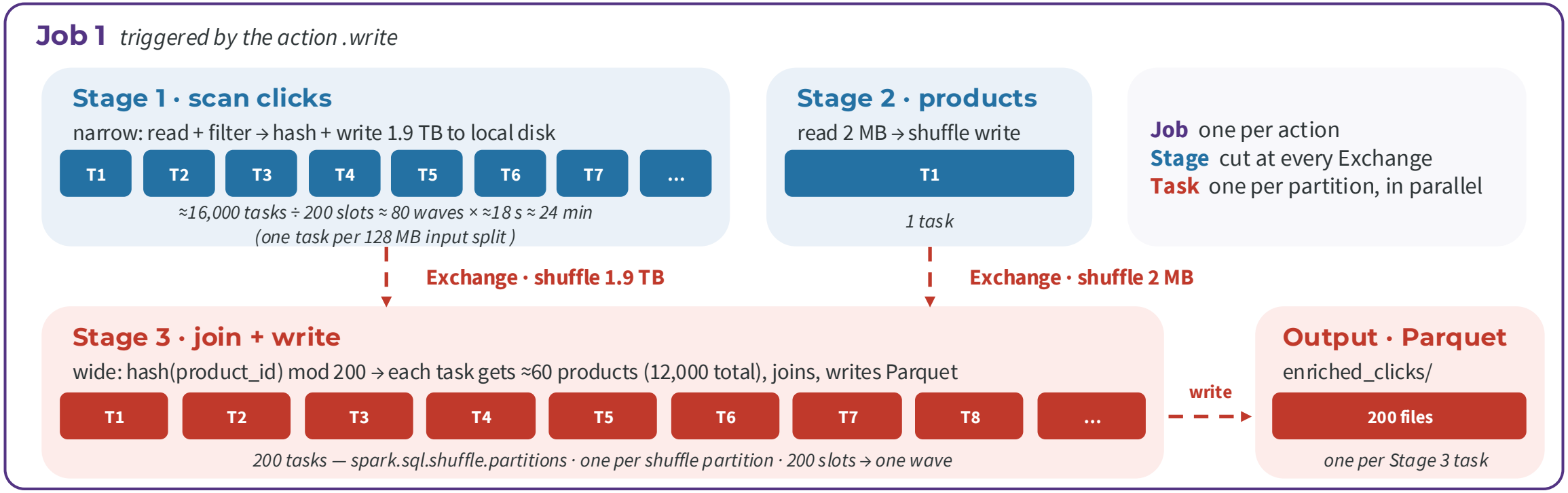
NEW TERMS

`show()` action: prints the first 20 rows

Scan read a table from storage
Skew work split unevenly across tasks (slide 27)

One Job, Three Stages

clicks.join(products, "product_id").write.parquet("enriched_clicks") — as the scheduler sees it



Read the tree, find the cost: *Stage 1 writes the 1.9 TB, Stage 3 reads and sorts it: both pay for the shuffle. Remove the shuffle, not the stage.*

NEW TERMS

- **`Input split`** a 128 MB block of input; one task each
- **`Shuffle partition`** one bucket of shuffle output
- **`spark.sql.shuffle.partitions`** their count (200)

groupBy Moves Counts, Not Rows

```
# count clicks per product
clicks.groupBy("product_id")
      .count().explain()
== Physical Plan ==
*(2) HashAggregate [count]
+- Exchange hashpart.(product_id, 200)
   +- *(1) HashAggregate [partial_count]
      +- FileScan parquet [product_id]
```

How to read it: bottom-up

- **FileScan** → a leaf: plans start at the scans
- **partial_count** → each task counts its own rows (stage 1)
- **Exchange** → a shuffle, so count these first; here ≤12,000 counts per task
- **count** → merges the partial counts per key (stage 2)

16,000 tasks × ≤12,000 counts ≈ a few GB shuffled, not 2 TB. Aggregations combine before the shuffle; a join cannot, so every row moves.

NEW TERMS

HashAggregate groups rows by key in a hash table

Partial aggregate per-task pre-count, before the shuffle

***(1) · *(2)** whole-stage codegen ids, split at the Exchange

Map-side combine shrink data before it is shuffled

join Moves Every Row

```
# what will Spark actually do?
clicks.join(products, "product_id")
      .write.parquet("enriched_clicks")
      .explain()
== Physical Plan ==
Execute InsertIntoHadoopFsRelation
+- *SortMergeJoin [product_id]
   :- *Sort [product_id]
   :  +- Exchange hashpart.(clicks)
   +- *Sort [product_id]
      +- Exchange hashpart.(products)
```

What each node does (AQE off)

- **Exchange (clicks)** → clicks hashed by product_id to 200 partitions (mod 200): 1.9 TB moved
- **Exchange (products)** → products hashed by product_id to 200 partitions: only 2 MB
- **Sort ×2** → each partition sorted by key; 9.5 GB per task vs ≈4 GB memory: ≈1 TB spill
- **SortMergeJoin** → one pass over both sorted sides, emitting matches

Spark moved and sorted 2 TB to meet a 2 MB table. Is there a join that leaves the big side where it is?

NEW TERMS

- **Sort** orders a partition's rows by key
- **hashpart.(k)** row → partition hash(k) mod 200
- **SortMergeJoin** merge two key-sorted inputs
- **InsertIntoHadoopFsRelation** writes the result files

Move 2 TB, or Move 2 MB

Who moves?
the solution space

Both sides move
`sort-merge · shuffle-hash`
Works for any sizes, but pays the full shuffle
✗ **our job: 2 TB moved and sorted**

Only the small side moves
`broadcast`
Copy it to every executor; the big side stays put. Needs one small side
✓ **products is 2 MB**

Who decides?
the components

The planner
`before running`
Size estimate vs the 10 MB threshold
✗ **no size via JDBC**

You
`in the code`
broadcast() or a join hint
✓ **our fix, slide 16**

AQE
`at runtime`
Re-decides from real sizes after each shuffle (slide 24)
✗ **disabled in our job**

Our job: the planner had no size and AQE was off, so only you could choose broadcast. Next: every strategy, and how Spark picks.

NEW TERMS

`Broadcast` copy a small table to every executor
`Planner` Catalyst's choice before the job runs
`AQE` Adaptive Query Execution: re-plans at runtime
`Hint` a note in the query that forces a strategy

Join Strategies at a Glance

The strategies Spark can pick

Sort-Merge
shuffle + sort both sides — the default

Broadcast Hash
ship the small table to every executor

Shuffle Hash
shuffle both, hash the smaller side

Broadcast NL
no equi-key: small table × every row

Cartesian
full cross product — last resort

How Spark decides — and how you influence it



Statistics
Table sizes and AQE runtime stats drive the optimiser's choice



Thresholds
autoBroadcastJoinThreshold — only 10 MB by default



Hints
/*+ BROADCAST(p) */ or .hint("merge") override the optimiser



Equi-keys
No equality predicate? Only nested-loop strategies remain

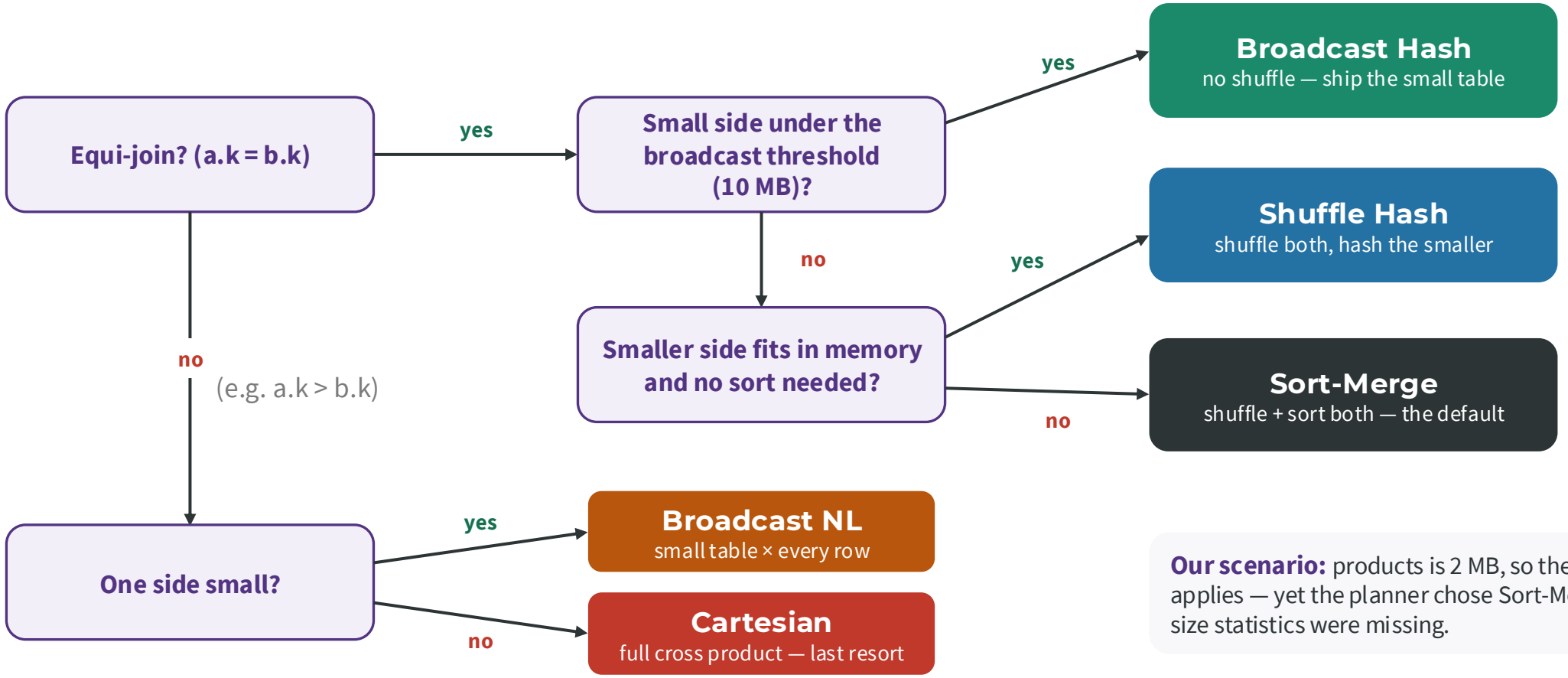
There is no universally best join — only the right join for these table sizes, this key distribution, this cluster.
Today you learn to choose deliberately.

NEW TERMS

- **`Broadcast`** send a full copy of a table to every executor
- **`Nested loop (NL)`** compare every row with every row
- **`Equi-key`** a join condition of the form a.k = b.k
- **`autoBroadcastJoinThreshold`** broadcast size cap
- **`Hint`** a note in the query that overrides the optimiser
- **`Statistics`** table sizes and row counts the optimiser uses

How Spark Picks the Join

The planner's decision path — and the knob you turn at each branch



Our scenario: products is 2 MB, so the top branch applies — yet the planner chose Sort-Merge because its size statistics were missing.

Your levers: *broadcast(df) or a threshold change forces the top branch; Shuffle Hash needs a /*+ SHUFFLE_HASH */ hint or preferSortMergeJoin = false, else Sort-Merge; AQE re-decides at runtime from real sizes.*

broadcast() Turns 40 Minutes into 4

```
# fixing the 40-minute join (no AQE)
from pyspark.sql.functions import broadcast
result = clicks.join(
    broadcast(products),  # 2 MB → ship it
    "product_id")

# verify — never trust, always check
result.explain()
*BroadcastHashJoin [product_id]
+- BroadcastExchange  # products only
```

Why it wins here

- **No shuffle** → 2 TB of clicks never leaves its executors
- **Map-side** → the join runs inside the scan stage
- **Cheap copy** → 2 MB × executors is trivial traffic
- **Watch out** → broadcasting a large table OOMs the driver
- **AQE** → auto-converts to broadcast when stats allow
- **Result** → 40 minutes becomes ~4

The clicks Exchange (1.9 TB) vanishes from the plan. One line changed, ~10× faster — because you knew which table was small and proved it in explain().

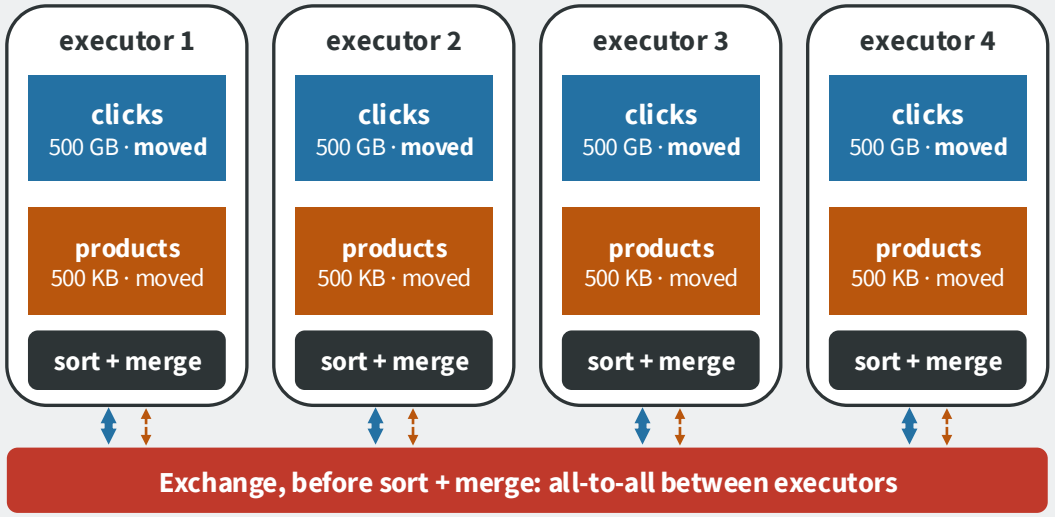
NEW TERMS

- **broadcast(df)** mark a DataFrame to be broadcast
- **BroadcastExchange** plan step that ships one table to all
- **Map-side join** join done in the scan stage, no shuffle
- **pyspark.sql.functions** Spark's built-in functions

Sort-Merge vs Broadcast Hash

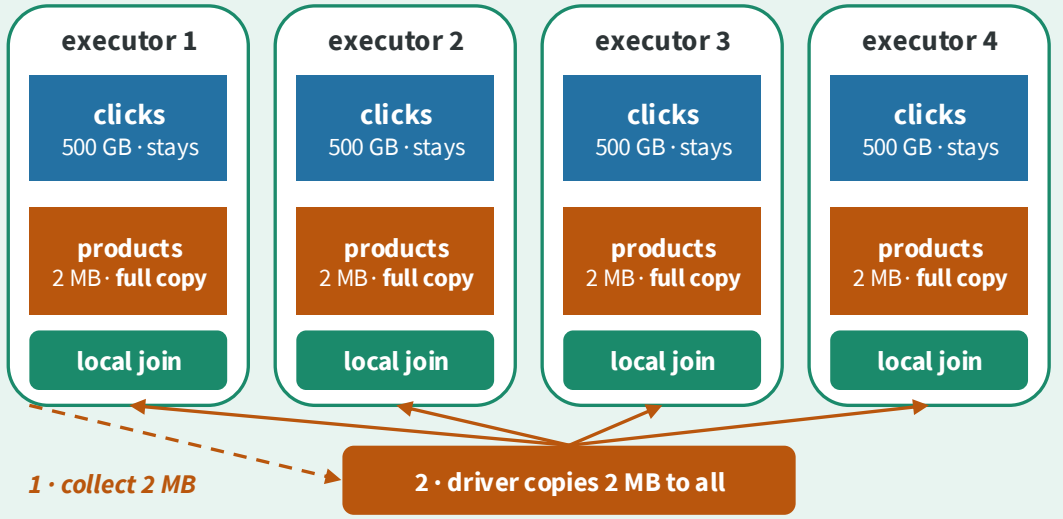
Same join, four executors — what crosses the network (arrows) and what stays put

Sort-Merge Join — the planner's default here



Network: ≈1.9 TB. Both tables hash-partitioned by product_id (hash(product_id) mod 200), then every partition sorted before merging.

Broadcast Hash Join — broadcast(products)



Network: 2 MB up + 4 × 2 MB ≈ 10 MB. The 2 TB of clicks never leave their executor; join runs inside the scan stage. The driver builds a hash table keyed by product_id.

Same rows out, *1.9 TB on the wire becomes ≈10 MB. Broadcast wins whenever one side fits comfortably in every executor's memory.*
On the real cluster (25 executors, slide 4): 2 MB up + 25 × 2 MB down ≈ 52 MB, still ≈35,000× less than 1.9 TB.

Sort-Merge vs Shuffle-Hash

When the small side is too big to broadcast (mid size), e.g. clicks (2 TB) × users (200 GB) — shuffle-hash

Sort-merge — the default

```
SortMergeJoin
  1. shuffle both sides
  2. sort: each partition by key
  3. merge: walk both in order
# scales to huge × huge
# spills to disk gracefully
# but pays the full sort
```

✓ **Robust, scales anywhere**

Shuffle-hash — no sort

```
ShuffledHashJoin
  1. shuffle both sides
  2. build: smaller → hash table
  3. probe: look up larger rows
# skips sorting entirely
# hash table must fit in
# executor memory — or OOM
```

✓ **Faster when memory allows**

Choosing — rules of thumb

```
one small → broadcast
one mid   → shuffle-hash
both huge → sort-merge
# ask; Spark obeys if it can:
clicks.join(
  users.hint("shuffle_hash"),
  "user_id")  # users → hash table
# then verify in explain()
```

✓ **Or let AQE decide at runtime**

Shuffle-hash needs each task's slice of the smaller table to fit in memory: users, 200 GB ÷ 800 partitions ≈ 250 MB.
Sort-merge is the safe default; shuffle-hash trades memory for speed. Skew breaks both — one giant key floods a single task; we return to it in the skew section (slides 27–29).

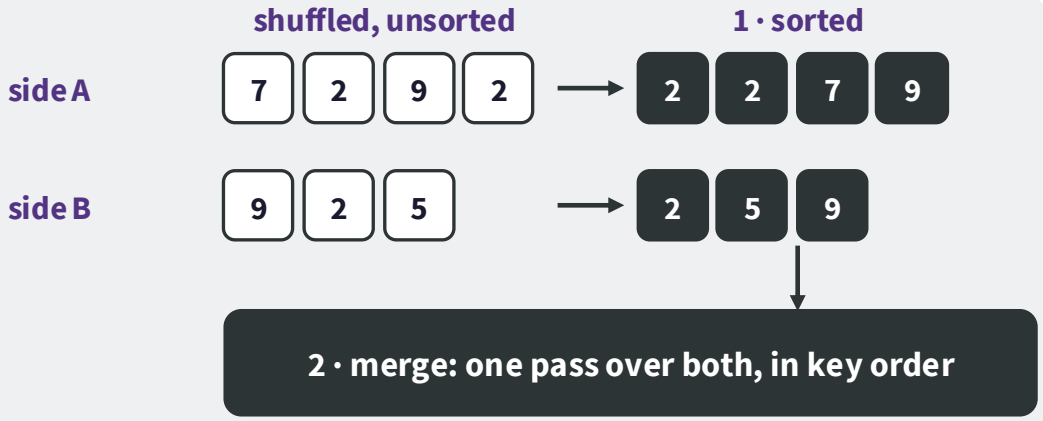
NEW TERMS

- **`ShuffledHashJoin`** plan name of a shuffle-hash join
- **`Sorted run`** a partition's rows in key order
- **`hint("merge")`** asks for a sort-merge join instead
- **`Hash table`** in-memory lookup from key to rows
- **`Probe`** look a row's key up in the hash table

Sort-Merge vs Shuffle-Hash, Drawn

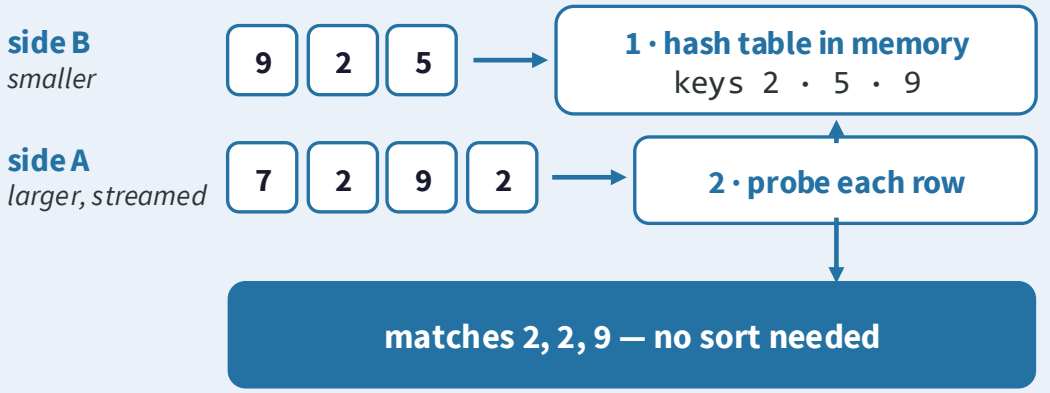
Inside one reduce task, after both sides were shuffled by key: the same rows, two ways to match them

Sort-Merge — sort both, then walk



Bounded memory: streams sorted runs and spills to disk when needed. The price is a full sort of both sides.

Shuffle-Hash — build, then probe



Memory-bound: skips the sort, but the whole hash table must fit in executor memory — or the task OOMs.

Same shuffle, different reduce work: *sort-merge pays CPU to stay safe on memory; shuffle-hash pays memory to skip the sort.*

NEW TERMS

- `Build side` the smaller side, loaded into the hash table
- `Stream side` the larger side, read row by row

Bucketing Turns a Range into a Key

"Tag each click with every campaign running when it happened": `clicks.ts BETWEEN c.start AND c.end  (Non-equi join)`

Without bucketing · BroadcastNestedLoopJoin

```
clicks.join(broadcast(campaigns),
  clicks.ts.between(campaigns.start, campaigns.end))
```

Every click tested against all 300 campaigns

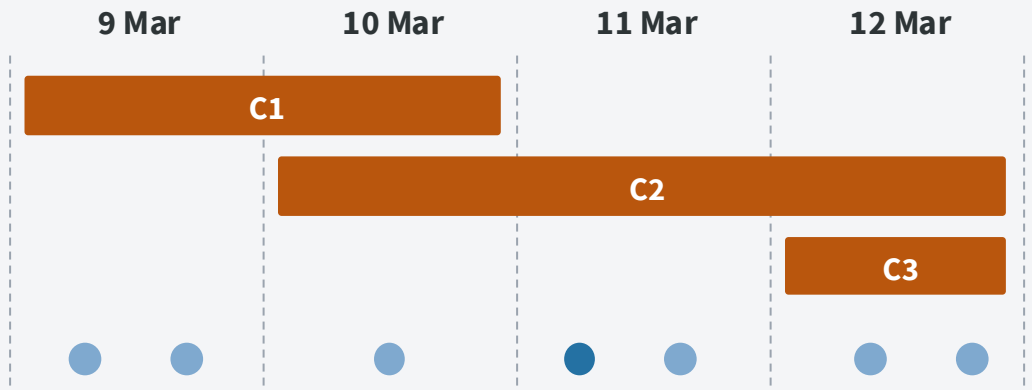
$4 \times 10^9 \times 300 = 1.2 \times 10^{12}$ tests

With bucketing · BroadcastHashJoin [day] + Filter

```
camp_days = campaigns.withColumn("day",
  explode(sequence(to_date("start"), to_date("end"))))
clicks.withColumn("day", to_date("ts"))
  .join(broadcast(camp_days), "day")
  .where("ts BETWEEN start AND end")
```

$4 \times 10^9 \times 3 = 1.2 \times 10^{10}$ tests, ≈100× fewer

Day buckets: a click only meets its day's campaigns



● clicks — the 11 Mar click is checked against C2 only, not all 300

camp_days (Mar): `C1·9` `C1·10` `C2·10` `C2·11` `C2·12` `C3·12`

300 campaigns × ~10 days ≈ 3,000 campaign-days, still broadcastable. Over ~3 years, ~3 are active per day: the day is the key (bucket).
The bucket gives Spark an equi-key; the filter keeps the result exact. BroadcastNestedLoopJoin = Broadcast NL.
Bucketing: pick a coarse key that every matching pair must share: day for timestamps, floor(price / 10) for prices, the /16 prefix for IP addresses, a grid cell for locations.

NEW TERMS

- **`Equi-key`** a join condition a.k = b.k Spark can hash or sort on
- **`Non-equi join`** any other condition: >, BETWEEN, OR
- **`Campaign`** a price promotion valid from *start* to *end*
- **`explode()`** one output row per array element
- **`Bucketing`** map a range to a coarse key to get an equi-join
- **`sequence(a,b)`** array of values from a to b, b included

Cartesian: Rows A × Rows B

No usable condition: every row of A paired with every row of B. Usually by accident.

1 · Forgotten condition

The join key is missing. Spark 3 runs it silently.

```
clicks.join(products)
```

$4 \times 10^9 \times 12{,}000 \approx 5 \times 10^{13}$ rows

Fix: add the key, "product_id"

2 · Intentional grid

Every day × every store, for a sales calendar with zeros.

```
days.crossJoin(stores)
```

365 × 500 = 182,500 rows

Fine: both sides are small

3 · No usable key

Neither side small, no equality:

```
abs(a.ts - b.ts) < 1800
similar(a.name, b.name)
```

(a UDF reading both tables)

2 TB × 2 TB: never finishes

Fix: add an equality, e.g. a.user_id = b.user_id

Warning signs: CartesianProduct in explain(), or output rows far above either input in the Spark UI.

NEW TERMS

- `CartesianProduct` plan node for a cross product
- `crossJoin()` asks for a Cartesian product on purpose
- `Self-join` a table joined with itself
- `crossJoin.enabled` true by default since Spark 3.0
- `UDF` user-defined function; reading both tables hides the key
- `Output rows` Spark UI SQL tab: rows each node emits

Spark UI — Reading the Evidence

THE THREE SPARK UI PAGES

1 — Jobs & Stages
how the job split, where time went

2 — Stage Detail
per-task times, shuffle read/write, spill

3 — SQL / DataFrame
the real plan, with runtime row counts

What the numbers say — the 40-minute join (slide 4)

```
# clicks.join(products, "product_id").write.parquet("enriched_clicks")
Stage 1  scan + shuffle write  ≈24 min  ← 80 waves × ≈18 s
Stage 2  scan products         ≈2 s     ← runs alongside stage 1
Stage 3  join + write
  Duration:     ≈16 min  ← job total ≈40 min
  Shuffle Read: 1.9 TB   ← both sides moved
  Spill (disk): ≈1 TB    ← 9.5 GB per task
  Tasks: 200/200 (0 failed)
    min 14 min  median 16 min  max 18 min
    ← max ≈ median: no skew, the plan is the cost
```

Habits of fast engineers

- ✓ Sort stages by duration — attack the biggest first
- ✓ Median vs max task time — your skew detector
- ✓ Shuffle read ≫ input size? Wrong join strategy
- ✓ Spills = memory pressure — repartition or resize
- ✓ Screenshot the UI — that is your lab evidence

Every performance claim this week — yours or the AI's — must point at a number on one of these screens.

NEW TERMS

`Shuffle Read` bytes a stage fetched from earlier shuffles
`Median` the middle value: half the tasks took longer
`Stage Detail` UI page with per-task metrics of one stage
`SQL tab` UI page with the run plan and row counts

Partition Tuning & AQE

```
# too few partitions = idle cores
# too many = overhead & tiny files
spark.conf.set(
  "spark.sql.shuffle.partitions", 800)

df.repartition(800, "key")  # shuffles
df.coalesce(100)  # merges, no shuffle

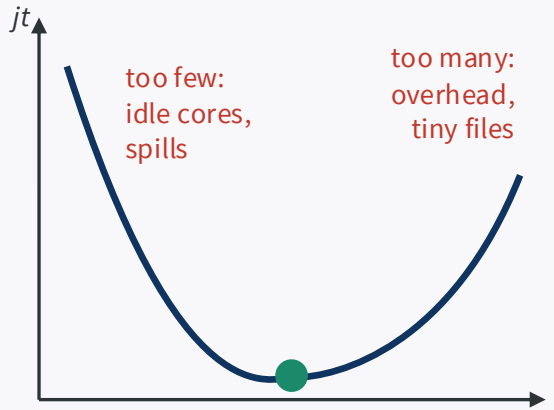
# or let runtime stats decide (default
since Spark 3.2)
"spark.sql.adaptive.enabled" = true
```

Rules of thumb

- **~128 MB** per partition → healthy task size
- **200** → default number of shuffle partitions: 1.9 TB ÷ 200 ≈ 9.5 GB per task
- **repartition** → full shuffle; sets up joins & writes
- **coalesce** → cheap merge; can only decrease
- **AQE** → coalesces & splits from real stats at runtime
- **Target** → tasks of seconds — not ms, not minutes

The sweet spot

job time vs. number of partitions



≈128 MB / partition

few → partitions → many

Partition count is the cheapest performance knob in Spark — and the first thing to check after the join strategy.
AQE makes it adaptive; it does not make it magic.

NEW TERMS

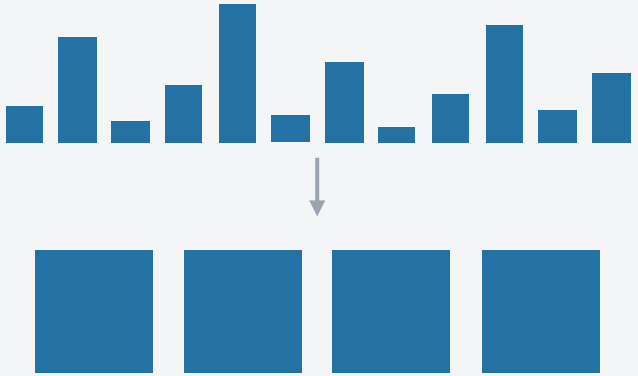
- **spark.conf.set** change a Spark setting for this session
- **repartition(n, col)** shuffle into n partitions by column
- **coalesce(n)** merge into fewer partitions, no shuffle
- **spark.sql.adaptive.enabled** switches AQE on

AQE Re-Plans After Every Shuffle

Stage runs → Shuffle written → **AQE reads real sizes** → Re-plans what's left

① Coalesce partitions

before: 200 partitions, mostly tiny



after: ~40 partitions of ≈128 MB

Fewer, healthier tasks; no tiny files

② Switch the join

plan before the run

`SortMergeJoin`
`Exchange ×2 · Sort ×2`

↓ **real size: products = 2 MB**

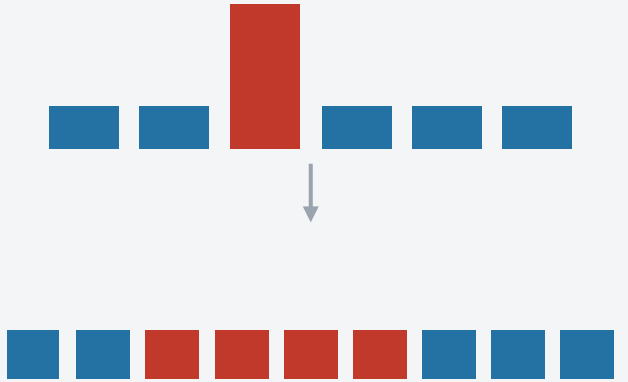
`BroadcastHashJoin`
`local shuffle read`

✓ **skips the network fetch and the sort**

✗ **the 1.9 TB shuffle write is already paid**

③ Split skew

before: one partition 10× the rest



after: the big one cut into 4 tasks

Rows copied to each piece → slide 27

AQE fixes the plan with real numbers, but only after a shuffle: data already moved stays moved. Read explain() first.

NEW TERMS

- `AdaptiveSparkPlan` plan root shown when AQE is on
- `Map output stats` real shuffle sizes AQE reads
- `Local shuffle read` read shuffle files without the network
- `Skew join` AQE splits an oversized partition

Caching — Spend Memory Wisely

Caching keeps a computed DataFrame on the executors, so later actions reuse it, not recompute it.

✓ When caching pays

- **Reuse** — several actions, a loop or branches
- **Cost** — recompute ≫ cache read (shuffle, JDBC, UDF)
- **Size** — filter & select first; it must fit
- **Placement** — last common step, after the join/agg

Once cached

→ **Lazy** — fills on the first action
→ **Verify** — Storage tab: size, fraction cached
→ **Release** — unpersist() after the last use

Memory	Mem + Disk	Disk
MEMORY_ONLY	MEMORY_AND_DISK `df.cache()` default	DISK_ONLY
SPEED ✓ Fastest	SPEED ✓ Fast, spills	SPEED ✗ Slowest
IF RAM RUNS SHORT ✗ Evicted → recomputed	IF RAM RUNS SHORT ✓ Spills to disk, no recompute	IF RAM RUNS SHORT ✓ Unaffected: uses no RAM
BEST FOR ✓ Hot, small data	BEST FOR ✓ Iterative jobs	BEST FOR ✓ Huge intermediates

Cache what you reuse, after the expensive step — an unused cache is just stolen executor memory.

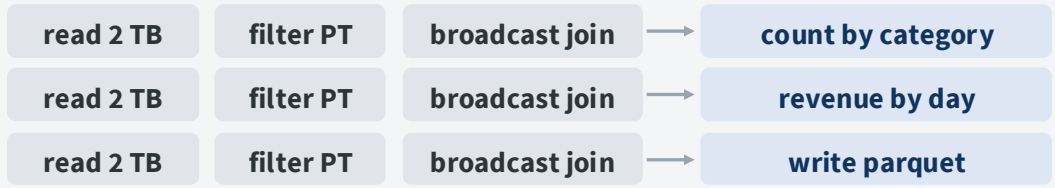
NEW TERMS

- `Storage level` where a cache lives: memory, disk or both
- `Materialise` compute and store the data for real
- `Evict` drop cached data to free memory
- `cache()` keep a DataFrame for reuse (= MEMORY_AND_DISK)
- `unpersist()` release a cached DataFrame
- `Storage tab` UI page listing cached data and its size

Cache Pays Only When Data Is Reused

Three actions on one dataset

Without cache



With cache



1 job reads and joins; actions 2–3 start from memory

```
enriched = clicks.filter(…).join(broadcast(products), "product_id").cache()
enriched.groupBy("category").count().show()       # action 1: fills cache
enriched.groupBy("day").sum("price").show()       # action 2: from cache
enriched.write.parquet("enriched_pt")             # action 3: from cache
enriched.unpersist()
```

An iterative loop

Without cache



10 reads from storage

With cache



1 read + 9 from memory

```
points = spark.read.parquet(…).cache()
for i in range(10):
    centers = update(points, centers)
    # update() ends in collect(): 1 action
points.unpersist()
```

← lab job 3: the cache-less loop

Used once? Don't cache. Used again? Filter first, cache after the expensive step, unpersist() when done.

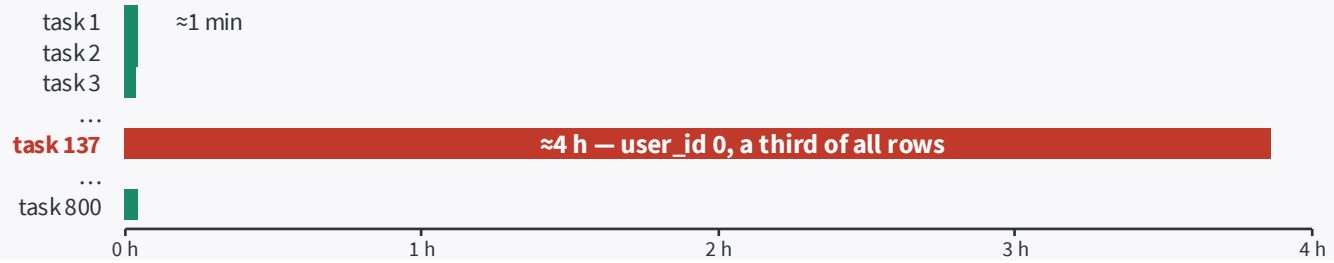
NEW TERMS

- **`Lineage`** the steps Spark replays to rebuild data
- **`Storage tab`** UI page listing cached DataFrames
- **`unpersist()`** drops a cached DataFrame
- **`Skipped stage`** a stage served from cache, not rerun

Data Skew — the Silent Killer

```
# "Enrich every click with its user"
# clicks 2 TB (4×10⁹) × users 200 GB (2×10⁸)
clicks.join(users, "user_id")
      .write.parquet("clicks_users")
# 800 partitions · users too big to broadcast
→ 799 tasks ≈1 min · 1 task ≈4 h
→ user_id 0 (guest) = ⅓ of rows ≈ 650 GB
```

Join stage task timeline *one bar per task, hours*



How to spot it

- **`max ≫ median`** task time — the one-straggler signature
- **`lopsided shuffle read`** — one task pulls far more data
- **`count your keys`** — groupBy(key).count().orderBy(desc) on a sample
- **`usual suspects`** — nulls, defaults, one giant customer

How to fix it

- **`AQE skew join`** — splits oversized partitions automatically
- **`salting`** — spread a hot key over N sub-keys, join on both
- **`broadcast`** — needs a small side; users is 200 GB
- **`isolate`** — treat null / hot keys separately, then union

Skew is why more nodes or more partitions fail — 799 task slots sit idle waiting for one key. Fix the data distribution, not the cluster size.

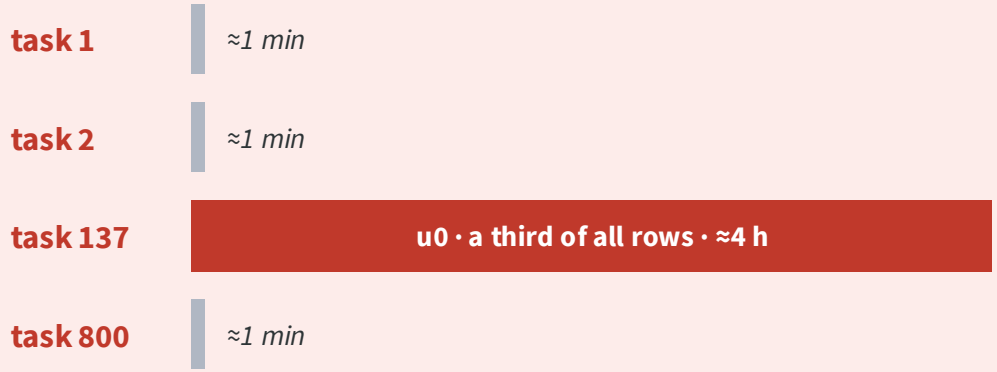
NEW TERMS

`Hot key` one key value holding a large share of rows
`Salting` add a computed suffix (e.g. hash mod N) so a hot key spreads out
`Skew join` AQE feature that splits oversized partitions
`union` stack two DataFrames with the same columns

Salting a Hot Key, Drawn

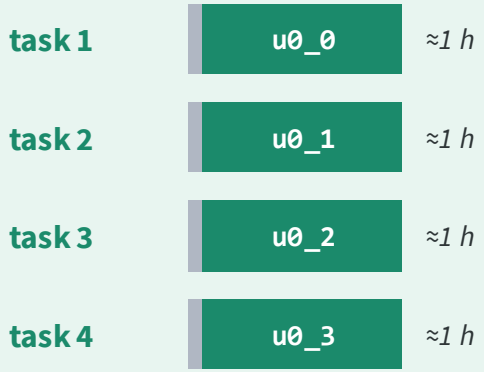
Clicks × users join, four of its 800 reduce tasks: bar length = time each task takes (1-min bars drawn as slivers)

Before — key u0 (guest) lands on one task



One key, one partition: hashing sends every u0 row to the same task, so 799 tasks finish and wait on task 137.

After — u0 salted into N = 4 sub-keys



Copy the small side N times: the guest user row becomes u0_0 … u0_3, so every salted click still finds its match.

Try AQE skew join first: *it splits hot join partitions automatically.*
Salt by hand when it cannot (e.g. full outer join); a larger N (e.g. 64) cuts 4 h to minutes.

NEW TERMS

- **u0** the guest user_id: every logged-out click
- **Sub-key** hot key + suffix: u0_0 … u0_3

Salting a Hot Key, in Code

Join · clicks × users, hot key user_id 0

```
from pyspark.sql import functions as F
N, hot = 4, [0]   # N salts per hot key; hot keys to salt
# 1. big side: spread the hot key's clicks over N salts
clicks_s = clicks.withColumn("salt",
    F.when(F.col("user_id").isin(hot),
           F.abs(F.hash("click_id")) % N)
     .otherwise(0))
# 2. small side: copy each hot-key user N times
users_s = users.withColumn("salt", F.explode(
    F.when(F.col("user_id").isin(hot),
           F.sequence(F.lit(0), F.lit(N - 1)))
     .otherwise(F.array(F.lit(0)))))
# 3. join on key + salt (kept once), then drop it
result = (clicks_s.join(users_s, ["user_id", "salt"])
                  .drop("salt"))
```

Check it worked

- **explain():** join keys [user_id, salt]
- **Spark UI:** max task ≈ median (4 h → minutes)
- **Rows out:** same count as without salting
- **Small side:** only the hot user is copied

Use it when

- **AQE is off,** or a full outer join
- **Not a groupBy count:** pre-counted (slide 11)

Hot key only: clicks get salt = hash(click_id) mod N; the hot user is copied N times (salt 0…N−1); join on user_id + salt. Others keep salt 0. Salt with hash(), not rand(): a retried task must get the same salts, or rows are lost or duplicated.

NEW TERMS

- `hash(col)` same input → same number, every run
- `isin(list)` true if the value is in the list
- `lit(x)` turns a Python value into a constant column
- `explode()` one output row per array element
- `sequence(a, b)` array from a to b (slide 20)
- `drop(col)` removes a column by name

Reading List

Ranked by priority — the primary set is what the lab assumes you have read; the secondary set goes deeper on the slides listed.

1 · Primary reading — read before the lab

1 Zaharia et al., *Resilient Distributed Datasets*, NSDI 2012
§2–5: lineage, wide dependencies, stage cuts · Slides 5–10

2 Armbrust et al., *Spark SQL*, SIGMOD 2015
§4 Catalyst: from logical plan to physical plan · Slides 11–13, 15

3 Chambers & Zaharia, *Spark: The Definitive Guide*, 2018
Ch. 8 Joins, ch. 19 Performance Tuning · Slides 14–26

4 Apache Spark docs, *Performance Tuning & AQE*, Spark 3.x
Broadcast threshold, coalescing, skew joins · Slides 15, 23–24, 27

2 · Secondary reading — go deeper

1 Dean & Ghemawat, *MapReduce*, OSDI 2004, §3
Where the shuffle comes from · Slides 7–8

2 Damji et al., *Learning Spark, 2nd ed.*, 2020, ch. 7
Spark UI walkthrough, tuning knobs · Slide 22

3 Kleppmann, *Designing Data-Intensive Applications*, ch. 10
Joins and skew in batch dataflow · Slides 14, 27–29

4 Databricks, *Adaptive Query Execution in Spark 3.0*, 2020
Why AQE re-plans joins and splits skew · Slides 23–24, 27

5 Apache Spark docs, *RDD Persistence*, Spark 3.x
Storage levels and when caching pays · Slides 25–26

6 Apache Spark docs, *Web UI*, Spark 3.x
Jobs, Stages and SQL tabs, task-time spread · Slide 22

Read the primary set in order; pick secondary items as your lab job needs.

PART 2 · HANDS-ON

Lab

05